

Database Concepts

Sumita Tyagi

Introduction

- **Data**-Any fact that can be recorded is called data.

E.g.- Texts, Numbers, Alphanumeric, Audio, Video, Images.

- **Database**- Collection of interrelated data is called database.
- **DBMS**-Database Management System- It is a collection of interrelated data and a set of programs to access those data. It helps us to store and retrieve database information in a convenient and efficient manner.
- **DBMS Software**- Ms Access, Oracle, SQL Server, MySQL, Sysbase

MS Excel- It has database management capability, but not a pure DBMS

Introduction

- **Languages used**-SQL(Structured Query Language),PL/SQL is a combination of SQL along with the procedural features of programming languages.
- **Applications**-Banking, Airline, Retail, Manufacturing, Universities.

Advantages of DBMS over File system

- **Data redundancy and inconsistency**

Redundancy is the concept of repetition of data i.e. each data may have more than a single copy. The file system cannot control redundancy of data as each user defines and maintains the needed files for a specific application to run. Whereas DBMS controls redundancy by maintaining a single repository of data that is defined once and is accessed by many users. As there is no or less redundancy, data remains consistent.

- **Data sharing**

File system does not allow sharing of data or sharing is too complex. Whereas in DBMS, data can be shared easily due to centralized system.

- **Data concurrency**

Concurrent access to data means more than one user is accessing the same data at the same time. Anomalies occur when changes made by one user gets lost because of changes made by other user. File system does not provide any procedure to stop anomalies. Whereas DBMS provides a locking system to stop anomalies to occur.

- **Data searching**

For every search operation performed on file system, a different application program has to be written. While DBMS provides inbuilt searching operations. User only have to write a small query to retrieve data from database.

- **Data integrity**

There may be cases when some constraints need to be applied on the data before inserting it in database. The file system does not provide any procedure to check these constraints automatically. Whereas DBMS maintains data integrity by enforcing user defined constraints on data by itself.

- **Data security**

A file system provides a password mechanism to protect the database but how longer can the password be protected? No one can guarantee that. This doesn't happen in the case of DBMS. DBMS has specialized features that help provide shielding to its data.

Relational Data Model

Table also called **Relation**

Domain
Ex: NOT NULL

Domain is pool of values from which actual values appearing in a column are taken.

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Tuple OR Row

Total # of rows is **Cardinality**

Column OR Attributes

Total # of column is **Degree**

Keys in DBMS

- **Primary Key**- It is an attribute or a combination of attribute that uniquely identifies a record.
- **Candidate Key**- It is an attribute or a combination of attributes that uniquely identifies a record and can be chosen to be primary key.
- **Alternate Key**- Candidate keys that are left out after choosing the primary key.
- **Foreign Key**- Foreign key is an attribute in table which refers to primary key of any other table.
- **Referential Integrity**- For referential integrity to hold in a relational database, any column in a base table that is declared a foreign key can only contain either null values or values from a parent table's primary key or a candidate key. In other words, when a foreign key value is used it must reference a valid, existing primary key in the parent table.

Keys in DBMS

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

Student Table

Student_ID	Stream	Marks	Department Code
1	Commerce	94	D002
2	Science	89	D001
3	Arts	76	D003

Subject Table

Here Student_ID is foreign key as it refers to StudID key in student table

Classification of SQL statements

- **DDL**- Data Definition Language actually consists of the SQL commands that can be used to define the database schema. It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database.

Examples of DDL commands:

- CREATE – is used to create the database or its objects (like table, index, function, views, store procedure and triggers).
- DROP – is used to delete objects from the database.
- ALTER-is used to alter the structure of the database.
- TRUNCATE—is used to remove all records from a table, including all spaces allocated for the records are removed.
- COMMENT —is used to add comments to the data dictionary.
- RENAME —is used to rename an object existing in the database.

Classification of SQL statements

- **DML**- Data Manipulation Language- The SQL commands that deals with the manipulation of data present in the database belong to DML or Data Manipulation Language and this includes most of the SQL statements.

Examples of DML:

- INSERT – is used to insert data into a table.
- UPDATE – is used to update existing data within a table.
- DELETE – is used to delete records from a database table.

Classification of SQL statements

- **DCL**- Data Control Language- DCL includes commands such as GRANT and REVOKE which mainly deals with the rights, permissions and other controls of the database system.

Examples of DCL commands:

- GRANT-gives user's access privileges to database.
- REVOKE-withdraw user's access privileges given by using the GRANT command.
- **TCL** - Transaction Control Language-TCL commands deals with the transaction within database.

Examples of TCL commands:

- COMMIT– commits a Transaction.
- ROLLBACK– rollbacks a transaction in case of any error occurs.
- SAVEPOINT–sets a savepoint within a transaction.

Data Types in MySQL

DATE TYPE	SPEC	DATA TYPE	SPEC
CHAR	String (0 - 255)	INT	Integer (-2147483648 to 2147483647)
VARCHAR	String (0 - 255)	BIGINT	Integer (-9223372036854775808 to 9223372036854775807)
TINYTEXT	String (0 - 255)	FLOAT	Decimal (precise to 23 digits)
TEXT	String (0 - 65535)	DOUBLE	Decimal (24 to 53 digits)
BLOB	String (0 - 65535)	DECIMAL	"DOUBLE" stored as string
MEDIUMTEXT	String (0 - 16777215)	DATE	YYYY-MM-DD
MEDIUMBLOB	String (0 - 16777215)	DATETIME	YYYY-MM-DD HH:MM:SS
LONGTEXT	String (0 - 4294967295)	TIMESTAMP	YYYYMMDDHHMMSS
LOBLOB	String (0 - 4294967295)	TIME	HH:MM:SS
TINYINT	Integer (-128 to 127)	ENUM	One of preset options
SMALLINT	Integer (-32768 to 32767)	SET	Selection of preset options
MEDIUMINT	Integer (-8388608 to 8388607)	BOOLEAN	TINYINT(1)

Create Database

- `SHOW DATABASES;`
- `CREATE DATABASE nameofdatabase;`
- `USE nameofdatabase;`

Constraints in a table

- **NOT NULL:** This constraint specifies that the column cannot have NULL values.
- **UNIQUE:** This constraint specifies that this particular column will have only unique values, though it allows to enter NULL for one record.
- **PRIMARY KEY:** This constraint specifies that this particular column will be primary key(having unique values), NULL is not an allowed value.
- **CHECK:** This constraint will check for a particular condition while entering data in database.
- **DEFAULT:** This constraint specifies the default value that the column will have if we don't specify any value.

- Write the SQL command to create a table named TOURNAMENT based on the following schema, ensuring all specified constraints are strictly applied:

Column Name	Data Type	Size / Format	Constraint / Description
GamerID	VARCHAR	6	Primary Key (e.g., 'G101')
GamerName	VARCHAR	30	Cannot be left blank (NOT NULL)
GameName	VARCHAR	25	Default value should be 'Minecraft'
Score	INT		Must be 100 or higher (CHECK)
Status	VARCHAR	15	Current status (e.g., 'Qualified', 'Eliminated')

CREATE TABLE COMMAND

DDL Command

```
CREATE TABLE TOURNAMENT (  
    GamerID VARCHAR(6) PRIMARY KEY,  
    GamerName VARCHAR(30) NOT NULL,  
    GameName VARCHAR(25) DEFAULT 'Minecraft',  
    Score INT CHECK (Score >= 100),  
    Status VARCHAR(15)  
);  
  
DESC TOURNAMENT;
```

INSERT COMMAND

DML Command

- **INSERT INTO TOURNAMENT**

VALUES ('G101', 'Alpha_Wolf', 'Valorant', 1250, 'Qualified');

- -- Omitting 'GameName' to test the **DEFAULT** constraint:

- **INSERT INTO TOURNAMENT** (GamerID, GamerName, Score, Status) **VALUES** ('G103', 'NeonKnight', 450, 'Qualified');

JOINS IN MySQL

Table: Club

ClubID	ClubName	Teacher_Incharge
C1	Robotics	Mr. Davis
C2	Drama	Ms. Vance
C3	Gaming	Mr. Ramirez
C4	Art	NULL

Table: Student

StudentID	Name	Grade	ClubID
101	Liam	12	C1
102	Sarah	11	NULL
103	Maya	12	C2
104	Alex	NULL	C1

Table Creation command for child table

```
CREATE TABLE Clubs (  
    ClubID VARCHAR(10) PRIMARY KEY,  
    ClubName VARCHAR(50) NOT NULL,  
    Teacher_Incharge VARCHAR(50) );
```

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Grade INT,  
    ClubID VARCHAR(10),  
    FOREIGN KEY (ClubID) REFERENCES Clubs(ClubID)  
    on update cascade on delete cascade);
```

EQUI JOIN

```
SELECT Students.Name, Clubs.ClubName, Students.ClubID  
FROM Students, Clubs  
WHERE Students.ClubID = Clubs.ClubID;
```